Case study

Code:

```
# Hidden Markov Model for user engagement
from hmmlearn import hmm
import numpy as np

# 3 hidden states: Interested, Neutral, Disengaged
model = hmm.CategoricalHMM(n_components=3, n_iter=100)

# Equal starting probability for all states
model.startprob_ = np.array([0.33, 0.33, 0.34])

# More balanced transition matrix
model.transmat_ = np.array([
    [0.5, 0.3, 0.2],  # Interested  → can transition to Neutral/Disengaged
    [0.2, 0.5, 0.3],  # Neutral     → can transition either way
    [0.1, 0.3, 0.6]   # Disengaged  → tends to stay disengaged
])

# Clearer distinction between states
model.emissionprob_ = np.array([
    [0.9, 0.1],  # Interested:  very likely to click
    [0.5, 0.5],  # Neutral:     equal chance
    [0.1, 0.9]   # Disengaged:  very unlikely to click
])

# Observed sequence: 0=click, 1=no-click
observations = np.array([[0, 1, 1, 0]]).T

hidden_states = model.predict(observations)
```

```
# Map state numbers to labels

state_labels = {0: "Interested", 1: "Neutral", 2: "Disengaged"}

labeled_states = [state_labels[s] for s in hidden_states]


print("Predicted Hidden States (raw)   :", hidden_states)

print("Predicted Hidden States (labeled):", labeled_states)
```

OUTPUT:

```
===== RESTART: C:\Users\CBIT\OneDrive\Documents\Desktop\099casestudyacc.py =====
Predicted Hidden States (raw)    : [0 2 2 1]
Predicted Hidden States (labeled): ['Interested', 'Disengaged', 'Disengaged', 'N
eutral']
```

Case study 2:

Student attention prediction

Code:

```
# Hidden Markov Model - Student Attention Prediction

# Hidden States : Attentive, Distracted, Sleeping

# Observations  : 0 = Responds, 1 = No Response


from hmmlearn import hmm

import numpy as np


# ————————————————————————————————————————

# 3 Hidden States:

#   0 = Attentive

#   1 = Distracted

#   2 = Sleeping

# ————————————————————————————————————————


model = hmm.CategoricalHMM(n_components=3, n_iter=100)


# Initial state probabilities

model.startprob_ = np.array([0.5, 0.3, 0.2])
```

```
# Transition probabilities
model.transmat_ = np.array([
    [0.6, 0.3, 0.1],  # Attentive  → mostly stays Attentive
    [0.3, 0.4, 0.3],  # Distracted → can go either way
    [0.2, 0.3, 0.5]   # Sleeping   → tends to stay Sleeping
])

# Emission probabilities
# 0 = Responds, 1 = No Response
model.emissionprob_ = np.array([
    [0.9, 0.1],  # Attentive  → very likely to respond
    [0.5, 0.5],  # Distracted → equal chance
    [0.1, 0.9]   # Sleeping   → very unlikely to respond
])

# ————————————————————————————————————————
# Observed sequence (6 questions asked)
# 0 = Responds, 1 = No Response
# ————————————————————————————————————————
observations = np.array([[0, 0, 1, 1, 0, 1]]).T

# Predict hidden attention states
hidden_states = model.predict(observations)

# ————————————————————————————————————————
# Decode results
# ————————————————————————————————————————
state_labels = {0: "Attentive",  1: "Distracted", 2: "Sleeping"}
obs_labels   = {0: "Responds  ", 1: "No Response"}
```

```
print("=" * 42)
print("  Student Attention Prediction using HMM")
print("=" * 42)
print(f"{'Q.No':<6} {'Observation':<15} {'Predicted State'}")
print("-" * 42)

for q, (obs, state) in enumerate(zip(observations.flatten(), hidden_states), start=1):
    print(f"Q{q:<5} {obs_labels[obs]:<15} {state_labels[state]}")

print("=" * 42)
print(f"\nRaw Hidden States : {hidden_states}")
```

OUTPUT:

```
==========================================
  Student Attention Prediction using HMM
==========================================
Q.No   Observation     Predicted State
------------------------------------------
Q1     Responds        Attentive
Q2     Responds        Attentive
Q3     No Response     Sleeping
Q4     No Response     Sleeping
Q5     Responds        Distracted
Q6     No Response     Sleeping
==========================================

Raw Hidden States : [0 0 2 2 1 2]
```